

```

# =====
# pdf_to_grayscale
# =====

def _grayscale_convert(input_path, output_path, custom_pages=None,
skip_images=False):
    try:
        doc = fitz.open(input_path)
        output_doc = fitz.open()
        total_pages = len(doc)

        pages_to_convert = set()
        if custom_pages:
            for part in custom_pages.split(','):
                part = part.strip()
                if not part: continue
                if '-' in part:
                    start, end = map(int, part.split('-'))
                    pages_to_convert.update(range(start - 1, end))
                else:
                    pages_to_convert.add(int(part) - 1)
        else:
            pages_to_convert = set(range(total_pages))

        converted_count = 0
        for i in range(total_pages):
            if i in pages_to_convert:
                temp_doc = fitz.open()
                temp_doc.insert_pdf(doc, from_page=i, to_page=i)
                temp_doc.recolor(colorspace=fitz.csGRAY)
                if not skip_images:
                    temp_doc.rewrite_images(set_to_gray=True)
                output_doc.insert_pdf(temp_doc, from_page=0,
to_page=0)

                temp_doc.close()
                converted_count += 1

```

```

        else:
            output_doc.insert_pdf(doc, from_page=i, to_page=i)

            sys.stderr.write(f"PROGRESS:{int(((i + 1) / total_pages) *
100)}%\n")

    output_doc.save(output_path, garbage=4, deflate=True)
    output_doc.close()
    doc.close()

    return {
        "success": True,
        "output": f"Successfully converted {converted_count} pages
to grayscale",
        "error": "",
        "pageCount": total_pages,
        "convertedPages": converted_count,
        "hasImages": True,
        "hasVectorGraphics": True
    }
except Exception as e:
    return {"success": False, "output": "", "error": str(e),
"pageCount": 0, "convertedPages": 0, "hasImages": False,
"hasVectorGraphics": False}

class pdf_to_grayscale:
    @staticmethod
    def main():
        args = {'input_path': None, 'output_path': None,
'custom_pages': None, 'skip_images': False}
        i = 1
        while i < len(sys.argv):
            arg = sys.argv[i]
            if arg == '--custom-pages' and i + 1 < len(sys.argv):
                args['custom_pages'] = sys.argv[i + 1].strip('')

```

```

        i += 2
    elif arg == '--skip-images':
        args['skip_images'] = True
        i += 1
    elif not arg.startswith('--'):
        if args['input_path'] is None:
            args['input_path'] = arg.strip(' ')
        elif args['output_path'] is None:
            args['output_path'] = arg.strip(' ')
        i += 1
    else:
        i += 1

    if args['input_path'] is None or args['output_path'] is None:
        print(json.dumps({"success": False, "error": "Usage:
        grayscale input.pdf output.pdf [--custom-pages \"1,2,3-6\"] [--skip-
        images]})))
        return 1

    result = _grayscale_convert(args['input_path'],
    args['output_path'], args['custom_pages'], args['skip_images'])
    print(json.dumps(result))
    return 0 if result["success"] else 1

```

